Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3555858.3555878

RESEARCH-ARTICLE

Imitating Playstyle with Dynamic Time Warping Imitation

MARK FERGUSON, University of York, York, North Yorkshire, U.K.

SAM DEVLIN, Microsoft Research, Redmond, WA, United States

DANIEL KUDENKO, Gottfried Wilhelm Leibniz University Hannover, Hannover, Niedersachsen, Germany

JAMES ALFRED WALKER, University of York, York, North Yorkshire, U.K.

Open Access Support provided by:

Gottfried Wilhelm Leibniz University Hannover

University of York

Microsoft Research

PDF Download
3555858.3555878.pdf
02 April 2026
Total Citations: 2
Total Downloads: 281

Published: 04 November 2022

Citation in BibTeX format

FDG22: 17th International Conference on the Foundations of Digital Games
September 5 - 8, 2022
Athens, Greece

https://doi.org/10.1145/3555858.3555878
ISBN: 9781450397957

Imitating Playstyle with Dynamic Time Warping Imitation

Mark Ferguson
University of York
York, UK
maf541@york.ac.uk

Sam Devlin
Microsoft Research
Cambridge, UK
sam.devlin@microsoft.com

Daniel Kudenko
Leibniz University
Hannover, Germany
kudenko@l3s.de

James Alfred Walker
University of York
York, UK
james.walker@york.ac.uk

ABSTRACT

Imitation learning has been demonstrated as a useful technique in automatic game testing and the development of believable Non-Player Characters (NPCs). However, imitation learning methods typically focus on learning a policy to complete a task without consideration about the playstyle used. In this work we consider the case where the task is to imitate a given playstyle. We defined a player's playstyle based on the strategies they use in order to complete the overall task. This has been achieved by rewarding a learning agent based on the similarity of the agent and demonstration trajectories, within a learnt representation space. This allows the playstyle to be learnt in levels that differ to the one the demonstrations were collected in.

CCS CONCEPTS

• **Computing methodologies** → **Learning from demonstrations**.

KEYWORDS

Imitation Learning, Playstyle Learning, Games

ACM Reference Format:
Mark Ferguson, Sam Devlin, Daniel Kudenko, and James Alfred Walker. 2022. Imitating Playstyle with Dynamic Time Warping Imitation. In *FDG '22: Proceedings of the 17th International Conference on the Foundations of Digital Games (FDG '22), September 5–8, 2022, Athens, Greece.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3555858.3555878

1 INTRODUCTION

Within the field of reinforcement learning (RL) [30] great strides have been made in completing complex tasks based on a supplied reward signal. Many of these techniques have been applied to the video game domain [2, 21, 22, 25, 29], as the game score could be used as the reward signal. However, for many tasks it is hard to design a suitable reward signal. Further, it has been shown a poorly defined reward signal can lead the RL algorithms to learning undesirable behaviours by 'hacking' the reward signal to maximise the reward in an unconsidered way [6, 27]. For these kinds of tasks, it can be beneficial to use Imitation Learning (IL) instead. This is because IL approaches learn a policy based on a set of demonstrations, which are often easier to generate compared to a reward function.

The task of learning a human-like playstyle is one of these and hence imitation learning is a suitable approach. This means a set of demonstrations are required for each playstyle you want to learn. It is suggested clustering analysis on the playerbase data [8, 9, 17] could be used to assemble the playstyle demonstration sets. The aim is then to take a set of demonstrations and learn to play with the given playstyle, even in situations not shown in the demonstrations. However, standard imitation learning methods commonly run into issues when attempting this.

This led to the development of our proposed imitation method called Dynamic Time Warping Imitation (DTWI). It allows the learning of a playstyle in a situation that differs from what is shown in the demonstrations, without assuming the availability of the player's actions in the demonstrations. This is achieved by training a RL agent that is rewarded based on the similarity between the agent and demonstration trajectories, within a learnt representation space. This agent also incorporates a system to control exploration based on a prediction of the similarity of the next 5-steps to the demonstrations.

This paper is organised in the following way. Firstly, Section 2 explores the relevant literature for imitation learning and its possible applications. Secondly, Section 3 will describe the proposed method that aims to allow the imitation of playstyles. After which, Section 4 will introduce the environment and level variants used within this work. Then Section 5 will introduce the set of playstyles imitation was performed on. Next, Section 6 will discuss the procedure to optimise the parameters for each of the algorithms used. Following this, Section 7 will show the results for the experiments conducted, in addition to a discussion about how the new technique stands up to standard baselines for imitation learning. Section 8 will provide the main conclusion and takeaway points from this work as well as layout possible future work.

2 RELATED WORK

2.1 Imitation Learning

Imitation learning is a paradigm that aims to learn a policy based on a set of given demonstrations instead of a provided reward signal. These methods can be split into either single-modal imitation or

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
FDG '22, September 5–8, 2022, Athens, Greece
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9795-7/22/09.
https://doi.org/10.1145/3555858.3555878

multi-modal imitation depending on whether the demonstration set is for a single or multiple tasks.

Within the multi-modal imitation literature [16, 23] methods have been developed to simultaneously learn multiple tasks, with a latent code to control that target task. For example, to control if the car turns right or left. Here we make a distinction between multi-task and multi-style. Multi-task learning generally utilises common skills to complete tasks, each of which often have different end states. While multi-style learning normally has a common overall goal but choose different strategies in order to complete it.

On the other hand, single-modal imitation aims to learn a single task. Although, the dataset in this work contains multiple playstyles and hence could be used in multi-modal imitation, the decision was made to focus on single-modal imitation within this work and transfer the technique into multi-modal imitation in future work. Due to this, below we will focus on the single-modal literature.

Behaviour Cloning (BC) [4, 28] can be thought as one of the simplest single-modal imitation methods. This method aims to learn a policy that successfully predicts the action taken, given an example state from the demonstration set. However, as the policy learns to map directly from state to action, the policy does not generalise to changes in state distribution. Further, it assumes the availability of the player's action within the demonstration set.

Although, this assumption can be removed by the use of Behaviour Cloning from Observation (BCO) [32]. This is achieved by learning an inverse model alongside the policy. The inverse model aims to predict the action that caused the transition between sequential states. This model can then be used to predict the actions used in the demonstrations and hence allow the standard BC approach to be applied. While this does address one of the issues with BC, the issue of not generalising to changes in state distribution still remains. In this work, this is expected to be a major issue because of the change in level layout between the demonstration and the training situation.

An alternative option is Inverse Reinforcement Learning (IRL) [26] which first aims to learn a reward function based on the provided demonstrations. Then an agent is trained using a standard reinforcement learning (RL) approach. These two steps are then repeated.

Generative Adversarial Imitation Learning (GAIL) [18] is an example of this kind of technique. It utilises Generative Adversarial Networks (GANs) [12], in order to train a discriminator to split examples from the demonstration set (real transitions) and examples from the generator (fake transitions). The discriminator is then used to provide a reward signal to train the generator on. In the initial implementation the discriminator takes state, action pairs. An alternative option, within a code implementation [35], is to train the discriminator on the state, action, next state triplets instead, this is called Generative Adversarial Imitation Learning - Triple (GAIL-T). Although, these methods help to improve the generalisation of the policy to unseen states, it also reintroduces the action availability assumption.

Further development of GANs based techniques led to the development of Generative Adversarial Imitation from Observation (GAIFO) [33], which removes the action availability assumption. This is achieved by training the discriminator on state, next state pairs. An alternative option, within a code implementation [35], is to train the discriminator on only the current state which is called Generative Adversarial Imitation from Observation - Single (GAIFO-S).

However, all the discussed GANs based methods potentially face issues based on the changes between the demonstrations and training levels. This is because the discriminator maybe able to easily split the fake and real transitions based on the different level layouts.

An alternative IRL approach attempts to generate a reward function based on a linear combination of a feature vector [1]. While this could not be applied directly, due to the image based observation, it could be applied to the learnt representation vector generated from the encoder. However, this could also run into issues as it is likely that there are many ways a reward function could be defined to explain the demonstrations, some of which won't transfer the playstyle to the other level variants. For example, if the agent is rewarded based on the route the player takes. In this case if an object the player needs is moved, then the defined reward function would fail to pick it up and hence the playstyle would not be maintained.

Another reward option is based on minimising the distance between the state, action distributions of the learnt policy and provided demonstrations [7]. Similar ideas have also been applied to minimise the distance between the state distribution only [11, 24]. Although, these methods do aim to deal with changes in the environment, whether that be start location or transition dynamics, they assume the required path is the same. Within this work changes in the environment may result in the demonstrated path being incorrect.

Overall, a number of imitation learning approaches have been developed based on a range of ideas, including adversarial networks, reward learning and distribution matching. However, each of these run into issues when attempting to learn a playstyle in a different situation then shown in the demonstrations, hence the requirement for our proposed technique.

Based on the reviewed literature, a set of baseline algorithms were chosen to compare to. Firstly, this includes BC to provide an example of how methods learning directly from state to action perform when the level layout changes. In addition, a number of GANs methods were also included, by adapting publicly available code [35]. Given the observation is partially observed it makes it harder for the discriminator to split the data purely based on the difference in level layouts for the majority of the transitions.

2.2 Imitation Learning Applications

The ability to successfully imitate a given playstyle has a couple of possible applications within the gaming industry. The first application is for the automatic testing of new game content. Previous literature [19, 20] explored defining a set of personas and creating agents to explore these personas within the game. However, in this case the personas were tailored around the game and hence required a level of game knowledge. Further they required high level game features.

This led to a set of more general personas that could be applied to any game [14, 15]. However, it is possible the general persona set does not fully cover all possible personas within a game. If our proposed imitation technique is applied post-clustering, a set of

game specific playstyles can be found without any game knowledge. This would allow the testing to focus on the type of playstyles expected from the playerbase, instead of testing being only driven by curiosity [13].

A second possible application is in the development of NPCs. This could be achieved by using the learnt playstyle policies to control different NPCs and hence give each of them a unique feel. Further, previous literature has suggested imitation based methods can lead to more believable NPCs [31], which could improve the player experience. Alternatively, cooperative NPCs could be trained with each of the imitated playstyles to generate an agent that can cooperate with a range of playstyles. It has been shown training with human imitating policies can improve the performance when paired with human players in the future [5].

The two applications above require the successful imitation of a given playstyle. Firstly, to tailor the automatic playtesting to playstyles existing in the playerbase without a deep understanding of the game. Secondly, to development NPCs that can utilise a given playstyle in multiple situations in a believable manner. This hence motivated the development of our proposed technique.

3 DYNAMIC TIME WARPING IMITATION (DTWI)

In this section we introduce the proposed Dynamic Time Warping Imitation (DTWI) algorithm. The architecture of the DTWI system is shown in Figure 1. Below is a breakdown of each part of the system and an explanation of its purpose.

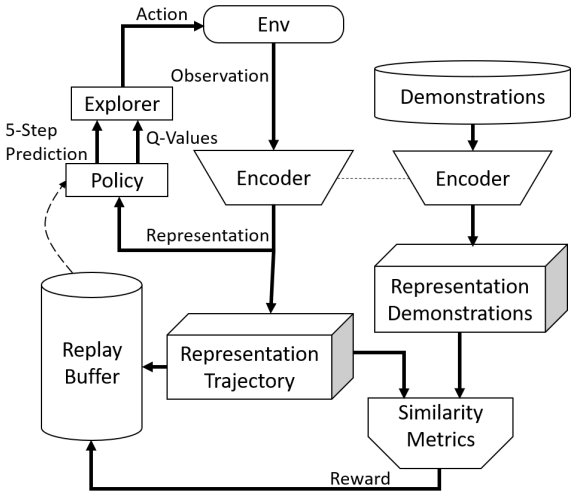


Figure 1: DTWI Architecture

3.1 Encoder

The first part of the system is the encoder. The aim was to pre-train an encoder with a useful representation space and then freeze it. It was decided to use Spatiotemporal DeepInfomax Variational Autoencoder (STDIM-VAE) [10], as it has been shown to allow the clustering of playstyles based on the learnt representations. This means if we reward based on the similarity of the trajectories within the representation space, we are rewarding based on the similarity of the playstyles.

The encoder was first utilised to convert the provided set of demonstrations into a set of representation demonstrations. These are then stored to allow the comparison between the agent's trajectories and the demonstrations at a later point.

The second use for the encoder is to convert the observation from the environment into a representation. This representation is stored in the representation trajectory buffer and is also passed to the policy. The policy network, which is made up of two dense layers each with 100 neurons, takes the representation and attempts to predict the Q-values and 5-Step Rewards.

3.2 Explorer

These two predictions are then passed to the explorer unit. The aim of this unit is to balance exploration and exploitation. This is achieved by the use of an adaptively chosen ϵ greedy approach. Due to the definition of the reward, discussed below, there is a maximum reward per step and hence a maximum 5-step reward value. This maximum reward is achieved by a perfect match for the next 5-steps. It is shown in Figure 2 that during training the 5-step reward prediction converges to the maximum reward value. Although, this happens faster for timesteps earlier in the episode.

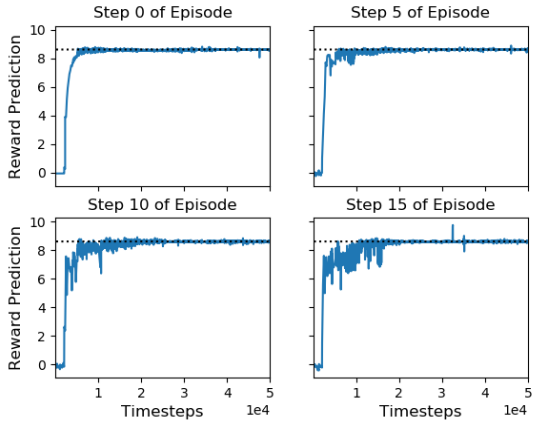


Figure 2: Reward Prediction (solid blue line) at Different Points in the Episode Compared against Theoretical Maximum Reward (dashed black line)

The prediction based ϵ value is chosen based on the difference between the predicted and max reward. Then the final ϵ is chosen based on the following rules. Firstly, for the first n steps the ϵ is held high to 0.9. This is to encourage more exploration at the beginning of the learning process. Following this the lowest value between the linearly decayed ϵ and prediction based ϵ is chosen. This ensures the ϵ value is decayed over the training process, but more exploitation is used as the predicted reward for the next 5-steps approaches the maximum reward value. Figure 3 shows an example of the ϵ value over the training process. Using these values, the explorer unit then outputs an action which is fed back to the environment. It can be seen as the reward prediction converges to the maximum

reward, the ϵ value converges to zero. This means the agent is able to exploit what has been learnt.

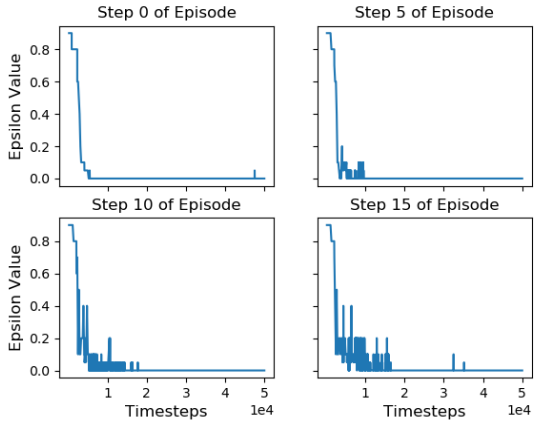


Figure 3: Epsilon Value (solid blue line) at Different Points in the Episode

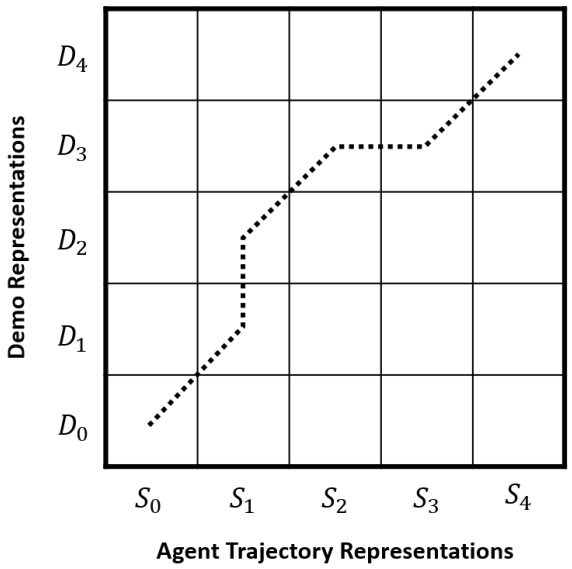


Figure 4: DTW Matching

3.3 Similarity Metric

The loop described above is run until an episode is completed and the whole agent representation trajectory is stored. After which the trajectory is compared to the representation demonstrations. The aim here is to use Dynamic Time Warping (DTW) to match each trajectory. In an ideal situation, the matching process would be applied to all demonstrations and the best match chosen. However, this process is slow especially if there is a large number of demonstrations. To speed up the matching, the similarity of the first representation from the agent's trajectory is compared to the first representation of all of the demonstrations. These demonstrations are filtered by the similarity of the representations. If this filtered set contains multiple demonstrations, this process is repeated with the next representation in the sequence until only one demonstration is left. The full DTW matching process is then applied to this demonstration. This results in a matching path between the representations. An example of this is shown in Figure 4 with the dotted line showing the matching path.

The next step is to convert the newly found matching path into a similarity reward for each state. The first component of this, is a measure of the cosine similarity of the matched representations. However, this needs to be compressed on to each agent state. This is achieved by taking the average similarity of each agent state and the demonstration state pair(s) that is matched on the path. For S_0, this is simply the cosine similarity between S_0 and D_0. Whereas for S_1, this is the average similarity of S_1 and D_1 and S_1 and D_2. The second component focuses on mismatches to encourage progression through the demonstration. This is achieved by penalising the agent if the path does not progress in both agent and demonstration representation sequences. In other words, the path does not go diagonally into the next square. For this component, a fixed penalty of 2.1 is given for any number of mismatches. This was chosen as the introduction of an additional step can give a maximum reward of 2 and hence the penalty must be higher than this. The overall reward, for the transition between S_x and S_{x+1}, is given as the similarity of S_{x+1} and the mismatch penalty of S_x. Additionally, the 5-step reward is calculated by a discounted sum of the next five rewards. However, if the end of the sequence is reached, the last reward is repeatedly used for future steps. This ensures the theoretical maximum reward is always possible.

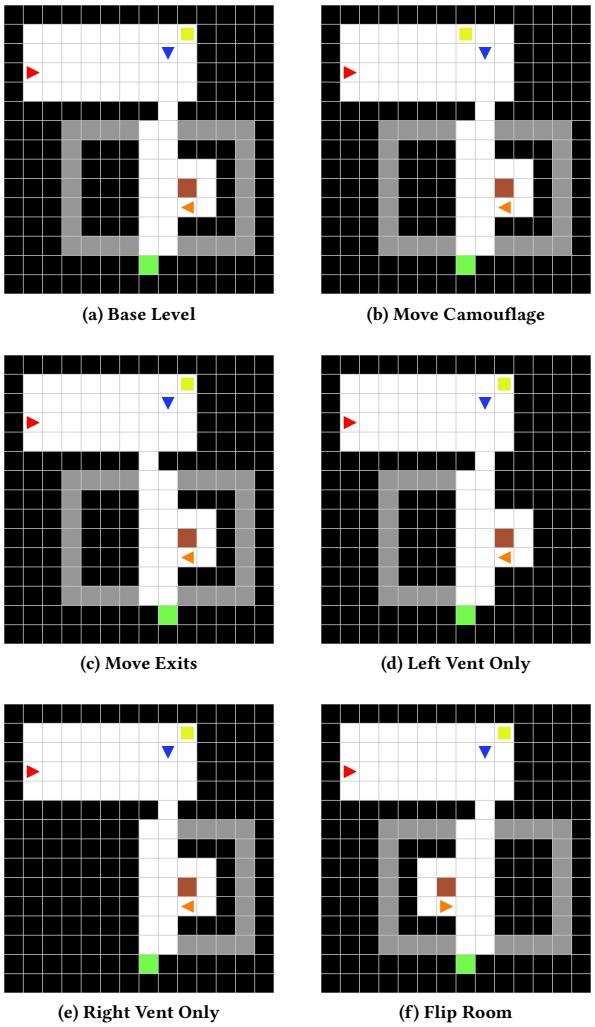
3.4 Replay Buffer

After the rewards and 5-step rewards are calculated for the episode trajectory, all these transitions are added into the replay buffer. Samples can then be pulled from the replay buffer in order to update the policy network using a standard RL approach.

4 ENVIRONMENT

Given this work focuses on the imitation of different playstyles, it is key the environment used allows the expression of a range of playstyles. It would also be beneficial if the playstyle set can be linked back to previous literature examining a game of the same genre. Based on this, a game in the 'stealth game' genre was designed as a grid-based environment. Within this environment a base level was designed and is shown in Figure 5a. In addition, a number of level variants were designed to allow the learning of the playstyle in different situations. The level variants are shown in Figures 5b-f.

The aim is for the player, represented by the blue triangle, to get to the green goal square. However, the player has to avoid two types of enemies. The first is represented by a red triangle which walks back and forth. The second enemy type is represented by an orange triangle. This enemy type stays stationary in their starting location. The player's objective is to avoid being caught. This is defined as being in 'line of sight' of an enemy. When it comes to these enemies, there are three ways to deal with them. Firstly, the



(a) Base Level (b) Move Camouflage

(c) Move Exits (d) Left Vent Only

(e) Right Vent Only (f) Flip Room

Figure 5: Level Variants

player can collect a yellow square, this represents picking up a camouflage package, meaning the player's triangle turns from blue to yellow and they are then no longer caught when in the enemies 'line of sight'. The second option is to 'takedown' the enemy. This is achieved by landing on the same space as the enemy without being caught. The third and final method of winning is to just avoid enemies entirely.

Additionally, a couple of visual choices were made. Firstly, light grey spaces aim to represent vent systems that you can often crawl through in stealth games. Secondly brown spaces represent an item within the room in which the player cannot go through.

5 PLAYSTYLES

As previously mentioned, the base level was designed to allow a range of playstyles. Three styles were chosen based on the 'Navigation Possibility Field' discussed in literature [34]. This means the playstyles used within this research were based on playstyles extracted from a related game in the stealth genre. The first style is defined as a 'Camouflage' playstyle and is based on the 'Roleplay Style' in the literature. In this style the player uses disguises to blend in and reach restricted areas. The second style is defined as a 'Takedown' playstyle which is based on the 'Brawn' playstyle in the literature. This means that the player uses close quarters combat in order to achieve their overall goal. The final style is defined as a 'Stealth' playstyle base on the similarly named style in the literature. In this case the player avoids the use of disguises and in addition, we have added the requirement that the player does not takedown any enemies. Each of these playstyles are defined in Table 1 based on a set of game variables. The camouflage game variable can either be 0 or 1 which relates to if the player has picked up the camouflage. The takedown variable tracks how many enemies the player has taken down. Then, the final completed game variable shows if the player has completed the level successfully. Further, each of the variant levels introduced above were designed in order to challenge the imitation of either one or all of the playstyles previously defined. For the 'Move Camouflage' level, the main challenge is for the 'Camouflage' playstyle, as it is required to detect the move of the camouflage and collect it from the new position. The 'Move Exit' level affects all playstyles as they all have to adjust their path to go through the moved exit. The 'Flip Room' level mainly affects the 'Takedown' playstyle as it is required to get behind the stationary enemy. Then finally the levels that isolate a single vent focus on challenging the 'Stealth' playstyle.

Table 1: Game Variables for each Playstyle

Playstyle	Camouflage	Takedowns	Completed
Camouflage	1.00	0.00	1.00
Takedown	0.00	2.00	1.00
Stealth	0.00	0.00	1.00

To evaluate the ability to imitate playstyles, firstly data needed to be generated for each of the previously discussed playstyles. To achieve this, a standard RL technique was used on a custom reward function. Through trial and error, it was found that the desired playstyles were produced by rewards based on the following events: collecting the camouflage, performing takedowns, use of vents, completing the level and being caught. The requirement of the use of trial and error in order to create the custom reward function, reinforces the difficulty of defining a custom function for a desired playstyle.

6 PARAMETER OPTIMIZATION

For each method, parameter tuning needs to be applied. This has been achieved by using the following process. Firstly, all parameters that need to be tuned are selected. Following this, depending on the parameter type, either a suitable set of discrete values or a range of values was selected. Next, tuning was conducted using a Bayesian based search strategy on the Weights & Bias platform (wandb) [3]. This was performed for a total of 120 runs shared between 3 agents running in parallel. During each run, at set intervals, the agent is evaluated based on the similarity of the agent's trajectories and the demonstrations and this evaluation is logged. After the sweep

is completed, all runs are sorted based on the similarity between the agent's trajectories and demonstrations. If there are multiple runs with the same similarity at the last logged evaluation, then the previous evaluation is used. This is repeatedly applied until all runs are ranked. From this ranking the top 5 parameter sets are chosen and for each of these sets, 3 repeats are run under that parameter set, rounded to 4 significant figures. After these repeats are completed, they are sorted by the same process as above. The top parameter set, based on the repeats, is then used in the experiments. This process was conducted for all the methods in this work and the resultant parameter sets can be found in Appendix A.

7 RESULTS

Presented below are the results from experiments conducted to imitate each of the three playstyles on each level variant. As mentioned above, each level variant is designed to challenge a certain playstyle, hence the analysis focuses on these combinations. An overview of the success of the imitation on these combinations can be seen in Figure 6. This analysis takes the form of a table which displays two game variables and completion rates which evaluate the playstyle being used. For each of these variables, the average of three training runs is shown at the completion of training. The ground truth (GT) for the playstyle is shown on the top row. Additionally, degree of success graphs are provided to evaluate the agent's ability not only to use the playstyle but use it to complete the level. Similarly, these are averaged over the three training runs.

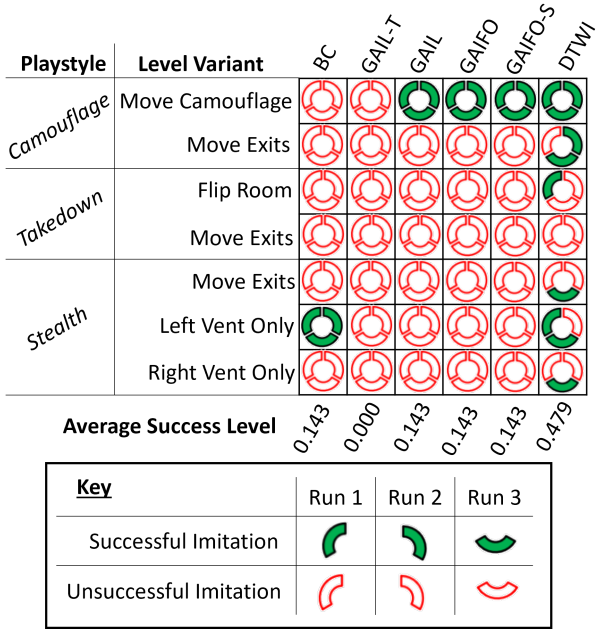


Figure 6: Imitation Success Overview

7.1 'Base' Level

The first set of experiments focus on the training on the base level, here the demonstration and training levels are the same. The final game variable values for the 'Camouflage', 'Takedown' and 'Stealth' playstyles can be found in Tables 2, 3, 4 respectively. When examined, it can be seen that the highest completion rate is found imitating the 'Camouflage' playstyle. This makes logical sense since the parameters were tuned for this combination of playstyle and level. Although, GAIL and GAIfO-S both have a slightly lower completion rate. Interestingly, these two GANs based methods both don't have access to the next state during training.

Table 2: Mean Game Variables for Imitating 'Camouflage' Playstyle on 'Base' Level (±SD)

Method	Camouflage	Takedowns	Completed
GT	1.00	0.00	1.00
BC	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00
GAIL-T	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00
GAIL	1.00 ± 0.00	0.00 ± 0.00	0.67 ± 0.47
GAIFO	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00
GAIFO-S	1.00 ± 0.00	0.00 ± 0.00	0.67 ± 0.47
DTWI	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00

Table 3: Mean Game Variables for Imitating 'Takedown' Playstyle on 'Base' Level (±SD)

Method	Camouflage	Takedowns	Completed
GT	0.00	2.00	1.00
BC	0.00 ± 0.00	2.00 ± 0.00	1.00 ± 0.00
GAIL-T	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIL	0.00 ± 0.00	1.00 ± 0.82	0.00 ± 0.00
GAIFO	0.00 ± 0.00	0.33 ± 0.47	0.00 ± 0.00
GAIFO-S	0.00 ± 0.00	2.00 ± 0.00	0.00 ± 0.00
DTWI	0.00 ± 0.00	1.67 ± 0.47	0.67 ± 0.47

Table 4: Mean Game Variables for Imitating 'Stealth' Playstyle on 'Base' Level (±SD)

Method	Camouflage	Takedowns	Completed
GT	0.00	0.00	1.00
BC	0.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00
GAIL-T	0.00 ± 0.00	0.00 ± 0.00	0.33 ± 0.47
GAIL	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIFO	0.00 ± 0.00	0.33 ± 0.47	0.00 ± 0.00
GAIFO-S	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
DTWI	0.00 ± 0.00	0.00 ± 0.00	0.33 ± 0.47

Further, Figure 7 shows the agents ability to utilise the correct playstyle to complete the level. It shows that most of the evaluated methods are able to learn the 'Camouflage' playstyle. However, only BC and DTWI have success on both of the other playstyles, but the success of DTWI is lower. In addition, this level of success is achieved by GAIL-T on the 'Stealth' playstyle as well. This shows that the BC method is a viable approach, if the demonstration and training state distributions are the same.

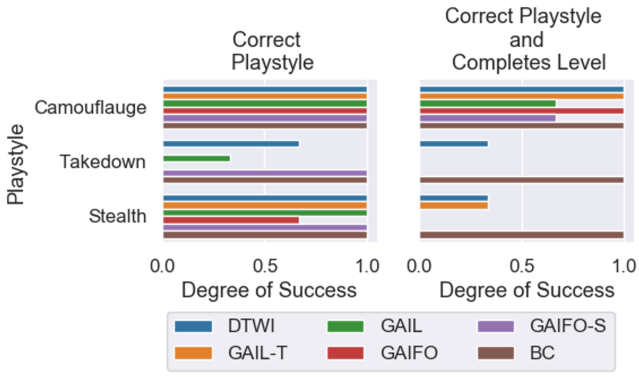


Figure 7: Success of Imitation on 'base' level

When accounting for the challenges of each playstyle, the success levels make more sense. For the 'Camouflage' playstyle, once the camouflage is picked up the player can simply walk through the rest of the level. On the other hand, for the 'Takedown' and 'Stealth' playstyles more skill is required. This takes the form of timing movements to either takedown or avoid NPCs and hence makes it harder to imitate.

7.2 'Move Camouflage' Level Variant

Next imitation was attempted on the 'Move Camouflage' variant. The final game variable values for the 'Camouflage' playstyle can be found in Table 5. From these results, it can be seen that moving the camouflage by two squares can cause major issues. This causes BC to never pick the camouflage item up and for GAIL-T to pick it up only once. However, it is noted when it does, the level is not completed. Conversely, all other methods are able to compensate for this change and complete the level with the correct playstyle, as shown in Figure 8.

Table 5: Mean Game Variables for Imitating 'Camouflage' Playstyle on 'Move Camouflage' Level Variant (±SD)

Method	Camouflage	Takedowns	Completed
GT	1.00	0.00	1.00
BC	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIL-T	0.33 ± 0.47	0.00 ± 0.00	0.00 ± 0.00
GAIL	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00
GAIFO	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00
GAIFO-S	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00
DTWI	1.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00

Interestingly, although this level was designed to mainly challenge the 'Camouflage' playstyle, it shows even small changes can cause issues for other methods, for example BC. Even on the 'Takedown' playstyle that does not require camouflage, the mere moving of it causes issues for its imitation.

7.3 'Flip Room' Level Variant

Following this, experiments were conducted on the 'Flip Room' variant. The final game variable values for the 'Takedown' playstyle

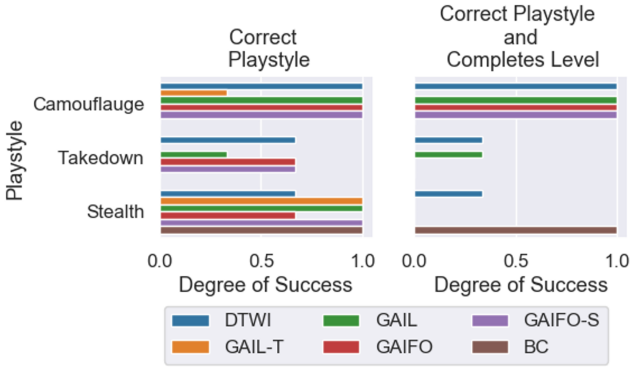


Figure 8: Success of Imitation on 'Move Camouflage' Level Variant

can be found in Table 6. From this it can be seen that only DTWI is able to complete the level, however the 'takedown' variable is lower than the ground truth for the 'Takedown' playstyle. This suggests some inconsistency in how the playstyle is imitated. This is shown in Figure 9, as only one run completed the level with the correct playstyle. The other two runs only performed 1 'takedown', compared to the 2 'takedowns' performed in the demonstrations.

Table 6: Mean Game Variables for Imitating 'Takedown' Playstyle on 'Flip Room' Level Variant (±SD)

Method	Camouflage	Takedowns	Completed
GT	0.00	2.00	1.00
BC	0.00 ± 0.00	1.00 ± 0.00	0.00 ± 0.00
GAIL-T	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIL	0.00 ± 0.00	0.33 ± 0.47	0.00 ± 0.00
GAIFO	0.00 ± 0.00	0.33 ± 0.47	0.00 ± 0.00
GAIFO-S	0.00 ± 0.00	1.00 ± 0.00	0.00 ± 0.00
DTWI	0.00 ± 0.00	1.33 ± 0.47	1.00 ± 0.00

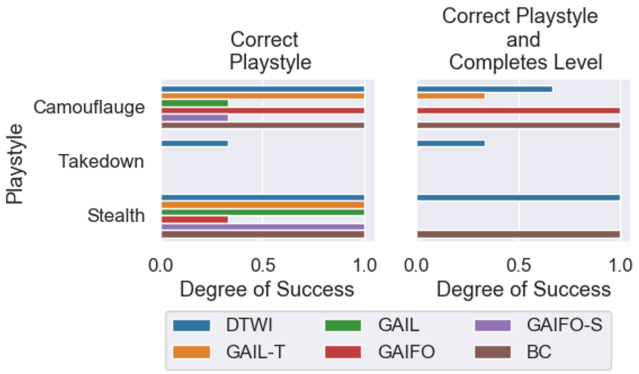


Figure 9: Success of Imitation on 'Flip Room' Level Variant

7.4 'Move Exits' Level Variant

Next, experiments were conducted on the 'Move Exits' variant. The final game variable values for the 'Takedown' playstyle can be seen in Table 7. Similarly, to the previous variant, only DTWI ever completes the level. However, similar inconsistencies can be seen in the playstyle of this run. Although it should be noted, only DTWI is able to successfully use the 'Camouflage' or 'Stealth' playstyle in order to complete this level variant, as shown in Figure 10

Table 7: Mean Game Variables for Imitating 'Takedown' Playstyle on 'Move Exits' Level Variant (±SD)

Method	Camouflage	Takedowns	Completed
GT	0.00	2.00	1.00
BC	0.00 ± 0.00	1.00 ± 0.00	0.00 ± 0.00
GAIL-T	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIL	0.00 ± 0.00	0.67 ± 0.94	0.00 ± 0.00
GAIFO	0.00 ± 0.00	0.67 ± 0.94	0.00 ± 0.00
GAIFO-S	0.00 ± 0.00	0.33 ± 0.47	0.00 ± 0.00
DTWI	0.00 ± 0.00	1.00 ± 0.82	0.33 ± 0.47

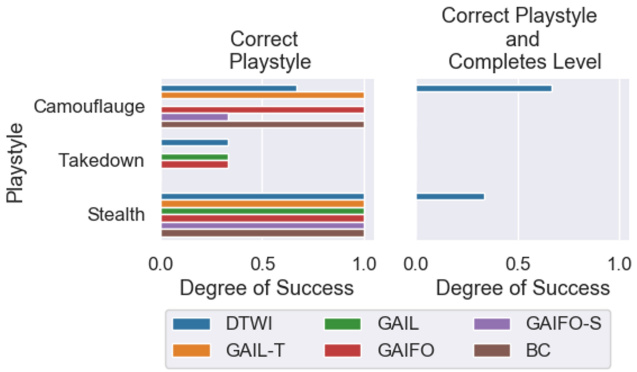


Figure 10: Success of Imitation on 'Move Exits' Level Variant

This further shows even minor changes can cause major issues with common imitation learning techniques. Although in these situations DTWI is often able to provide some level of success.

7.5 'Left Vent Only' Level Variant

We next consider imitating the 'Stealth' playstyle on the 'Left Vent Only' variant and the final game variable values here are displayed in Table 8. It should be noted that the demonstrations for the 'Stealth' playstyle all use the left vent and hence it was expected to have little effect on the ability to imitate the playstyle. This was found to be true as the same set of methods, minus GAIL-T, were successful on this level variant compared to the 'base' level.

This shows even removing unused parts of the map can cause issues for methods, for example GAIL-T, when imitating the 'Stealth' playstyle.

Table 8: Mean Game Variables for Imitating 'Stealth' Playstyle on 'Left Vent Only' Level Variant (±SD)

Method	Camouflage	Takedowns	Completed
GT	0.00	0.00	1.00
BC	0.00 ± 0.00	0.00 ± 0.00	1.00 ± 0.00
GAIL-T	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIL	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIFO	0.00 ± 0.00	0.33 ± 0.47	0.00 ± 0.00
GAIFO-S	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
DTWI	0.00 ± 0.00	0.00 ± 0.00	0.67 ± 0.47

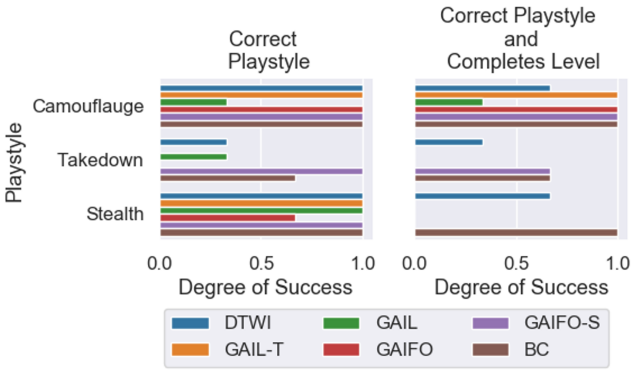


Figure 11: Success of Imitation on 'Left Vent Only' Level Variant

7.6 'Right Vent Only' Level Variant

Finally, the 'Stealth' playstyle was trained on the 'Right Vent Only' variant. Here the final game variable values can be seen in Table 7. In this case, BC was also unsuccessful and when investigated it appears that the agent stands in front of where the left vent should be. However, DTWI is still able to successfully complete the level, but only on one of the training runs, as shown in Figure 12.

Table 9: Mean Game Variables for Imitating 'Stealth' Playstyle on 'Right Vent Only' Level Variant (±SD)

Method	Camouflage	Takedowns	Completed
GT	0.00	0.00	1.00
BC	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIL-T	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIL	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIFO	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
GAIFO-S	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
DTWI	0.00 ± 0.00	0.00 ± 0.00	0.33 ± 0.47

In this case the vent used within the 'Stealth' demonstrations is removed. However, DTWI is able to recognise using the right vent would still be the same playstyle and hence learns to use it. Although, this is not consistent across all training runs.

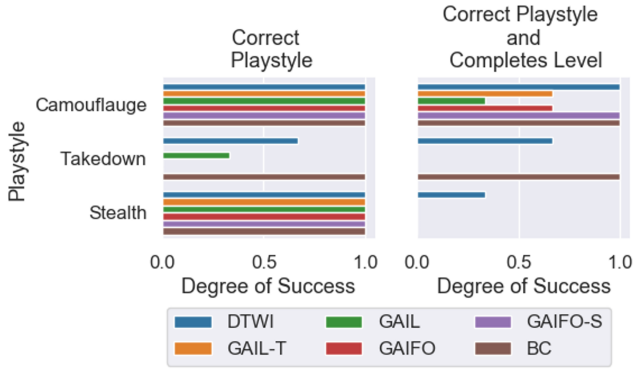


Figure 12: Success of Imitation on 'Right Vent Only' Level Variant

8 CONCLUSION AND FUTURE WORK

In this work we have presented a new imitation learning method Dynamic Time Warping Imitation (DTWI). This technique allows the imitation of a demonstrated playstyle from observation of states alone. This has been achieved by training on a reward function extracted from the similarity of the agent trajectories and the demonstrations using DTW in a learnt representation space.

When considering the imitation of the playstyle(s) that the level variants were designed to challenge, it has been shown on average our DTWI method outperforms the standard imitation learning baselines.

Following on from this work, there are a few possible avenues for future work. Firstly, to further examine the robustness of the proposed technique by experimenting on additional game environments. Secondly, to move to multi-modal imitation and evaluate if this is more data efficient than learning each playstyle individually. Finally, the learnt policy(s) could be evaluated for automated game testing or NPC development.

ACKNOWLEDGMENTS

This work was supported by Microsoft Research PhD Scholarship Programme.

REFERENCES

[1] Pieter Abbeel and Andrew Y Ng. 2004. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the twenty-first international conference on Machine learning*. 1.

[2] Adrià Puigdomènech Badia, Bilal Piot, Steven Kapturowski, Pablo Sprechmann, Alex Vitvitskyi, Zhaohan Daniel Guo, and Charles Blundell. 2020. Agent57: Outperforming the atari human benchmark. In *International Conference on Machine Learning*. PMLR, 507–517.

[3] Lukas Biewald. 2020. Experiment Tracking with Weights and Biases. https://www.wandb.com/ Software available from wandb.com.

[4] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Prasoon Goyal, Lawrence D Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, et al. 2016. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316* (2016).

[5] Micah Carroll, Rohin Shah, Mark K Ho, Tom Griffiths, Sanjit Seshia, Pieter Abbeel, and Anca Dragan. 2019. On the utility of learning about humans for human-ai coordination. *Advances in Neural Information Processing Systems* 32 (2019), 5174–5185.

[6] Jack Clark and Dario Amodei. 2016. Faulty reward functions in the wild. *Internet: https://blog. openai. com/faulty-reward-functions* (2016).

[7] Robert Dadashi, Léonard Hussenot, Matthieu Geist, and Olivier Pietquin. 2021. Primal Wasserstein Imitation Learning. In *ICLR 2021-Ninth International Conference on Learning Representations*.

[8] Anders Drachen, James Green, Chester Gray, Elie Harik, Patty Lu, Rafet Sifa, and Diego Klabjan. 2016. Guns and guardians: Comparative cluster analysis and behavioral profiling in destiny. In *2016 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 1–8.

[9] Anders Drachen, Rafet Sifa, Christian Bauckhage, and Christian Thurau. 2012. Guns, swords and data: Clustering of player behavior in computer games in the wild. In *2012 IEEE conference on Computational Intelligence and Games (CIG)*. IEEE, 163–170.

[10] Mark Ferguson, Sam Devlin, Daniel Kudenko, and James Alfred Walker. 2020. Player style clustering without game variables. In *International Conference on the Foundations of Digital Games*. 1–4.

[11] Tanmay Gangwani and Jian Peng. 2019. State-only Imitation with Transition Dynamics Mismatch. In *International Conference on Learning Representations*.

[12] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial nets. *Advances in neural information processing systems* 27 (2014).

[13] Camilo Gordillo, Joakim Bergdahl, Konrad Tollmar, and Linus Gisslén. 2021. Improving Playtesting Coverage via Curiosity Driven Reinforcement Learning Agents. In *2021 IEEE Conference on Games (CoG)*. IEEE, 1–8.

[14] Cristina Guerrero-Romero, Annie Louis, and Diego Perez-Liebana. 2017. Beyond playing to win: Diversifying heuristics for GVGAI. In *2017 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 118–125.

[15] Cristina Guerrero-Romero, Simon M Lucas, and Diego Perez-Liebana. 2018. Using a team of general ai algorithms to assist game design and testing. In *2018 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 1–8.

[16] Karol Hausman, Yevgen Chebotar, Stefan Schaal, Gaurav Sukhatme, and Joseph J Lim. 2017. Multi-modal imitation learning from unstructured demonstrations using generative adversarial nets. *Advances in neural information processing systems* 30 (2017).

[17] Roy Hayes and Peter Beling. 2018. Unsupervised Hierarchical Clustering of Build Orders in a Real-Time Strategy Game. *The Computer Games Journal* 7, 1 (2018), 5–26.

[18] Jonathan Ho and Stefano Ermon. 2016. Generative adversarial imitation learning. *Advances in neural information processing systems* 29 (2016), 4565–4573.

[19] Christoffer Holmgård, Michael Cerny Green, Antonios Liapis, and Julian Togelius. 2018. Automated playtesting with procedural personas through MCTS with evolved heuristics. *IEEE Transactions on Games* 11, 4 (2018), 352–362.

[20] Christoffer Holmgård, Antonios Liapis, Julian Togelius, and Georgios N Yannakakis. 2014. Evolving personas for player decision modeling. In *2014 IEEE Conference on Computational Intelligence and Games*. IEEE, 1–8.

[21] M Jaderberg, WM Czarnecki, I Dunning, L Marris, G Lever, AG Castaneda, et al. 2018. Human-level performance in first-person multiplayer games with population-based deep reinforcement learning. arXiv. *arXiv preprint arXiv:1807.01281* (2018).

[22] Max Jaderberg, Volodymyr Mnih, Wojciech Marian Czarnecki, Tom Schaul, Joel Z Leibo, David Silver, and Koray Kavukcuoglu. 2016. Reinforcement learning with unsupervised auxiliary tasks. *arXiv preprint arXiv:1611.05397* (2016).

[23] Yunzhu Li, Jiaming Song, and Stefano Ermon. 2017. Infogail: Interpretable imitation learning from visual demonstrations. *Advances in Neural Information Processing Systems* 30 (2017).

[24] Fangchen Liu, Zhan Ling, Tongzhou Mu, and Hao Su. 2019. State Alignment-based Imitation Learning. In *International Conference on Learning Representations*.

[25] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *nature* 518, 7540 (2015), 529–533.

[26] Andrew Y Ng, Stuart J Russell, et al. 2000. Algorithms for inverse reinforcement learning.. In *Icml*, Vol. 1. 2.

[27] Jette Randløv and Preben Alstrøm. 1998. Learning to Drive a Bicycle Using Reinforcement Learning and Shaping.. In *ICML*, Vol. 98. Citeseer, 463–471.

[28] Claude Sammut, Scott Hurst, Dana Kedzier, and Donald Michie. 1992. Learning to fly. In *Machine Learning Proceedings 1992*. Elsevier, 385–393.

[29] Kun Shao, Zhentao Tang, Yuanheng Zhu, Nannan Li, and Dongbin Zhao. 2019. A survey of deep reinforcement learning in video games. *arXiv preprint arXiv:1912.10944* (2019).

[30] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.

[31] Fabien Tencé, Cédric Buche, Pierre De Loor, and Olivier Marc. 2010. The challenge of believability in video games: Definitions, agents models and imitation learning. *arXiv preprint arXiv:1009.0451* (2010).

[32] Faraz Torabi, Garrett Warnell, and Peter Stone. 2018. Behavioral cloning from observation. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*. 4950–4957.

[33] Faraz Torabi, Garrett Warnell, and Peter Stone. 2018. Generative adversarial imitation from observation. *arXiv preprint arXiv:1807.06158* (2018).

[34] Anders Tychsen and Alessandro Canossa. 2008. Defining personas in games using metrics. In *Proceedings of the 2008 conference on future play: Research, play, share*. 73–80.
[35] Zhuangdi Zhu, Kaixiang Lin, Bo Dai, and Jiayu Zhou. 2020. Off-policy imitation learning from observations. *Advances in Neural Information Processing Systems* 33 (2020), 12402–12413.

A PARAMETER SELECTION

Below we will provide the parameter sets found via tuning for each of the methods in Tables 10-15. Further parameters were fixed through tuning. One such parameter is the policy network architecture. This was fixed to contain two hidden layers, each with a size of 100, for all methods. In addition, the batch size was fixed to 32 for DTWI.

Initial experiments showed the agent having trajectories vastly longer than the demonstration provided, caused issues with the GAIL-T imitation. Generally, in imitation work the episode lengths are not variable and hence this is not an issue. To solve this for the GANs based methods the maximum episode length was restricted to be close to the length of the provided demonstrations.

It should be noted the parameters were tuned to imitate the 'Camouflage' playstyle on the 'base' level, but also used on the other playstyles and level variants.

Table 10: Parameters for BC

Parameter	Value
Batch Size	32
Learning Rate	0.0001298

Table 11: Parameters for DTWI

Parameter	Value
Batch Accumulator	Mean
Double	True
Dueling	False
End Epsilon	0.1
Epsilon Check Step	5
Epsilon Max Step Ratio	0.9
Epsilon Min Steps	10
Gamma	0.9256
Head Freeze	2
Learning Rate	0.0004751
Prioritized	False
Replay Start Size	2000
Target Update	5000

Table 12: Parameters for GAIL

Parameter	Value
Adversary Entropy Coefficient	0.007580
Adversary Gradient Coefficient	15
Adversary Learning Rate	0.0008885
Dampening Factor	0.02543
Entropy Coefficient	0.006351
Gamma	0.9694
CG Iterations	15
KL Threshold	0.01706
GAE Factor	0.9651
Learning Rate	0.0008076
Timesteps per Batch	512
VF Iterations	5

Table 13: Parameters for GAIL-T

Parameter	Value
Adversary Entropy Coefficient	0.001100
Adversary Gradient Coefficient	15
Adversary Learning Rate	0.0008045
Dampening Factor	0.01459
Entropy Coefficient	0.009801
Gamma	0.9184
CG Iterations	20
KL Threshold	0.02914
GAE Factor	0.9727
Learning Rate	0.0002322
Timesteps per Batch	1024
VF Iterations	5

Table 14: Parameters for GAIFO

Parameter	Value
Adversary Entropy Coefficient	0.004170
Adversary Gradient Coefficient	20
Adversary Learning Rate	0.0002984
Dampening Factor	0.006979
Entropy Coefficient	0.003230
Gamma	0.9643
CG Iterations	20
KL Threshold	0.002455
GAE Factor	0.9720
Learning Rate	0.0008781
Timesteps per Batch	2048
VF Iterations	5

Table 15: Parameters for GAIFO-S

Parameter	Value
Adversary Entropy Coefficient	0.006498
Adversary Gradient Coefficient	15
Adversary Learning Rate	0.0005939
Dampening Factor	0.01289
Entropy Coefficient	0.006248
Gamma	0.9635
CG Iterations	20
KL Threshold	0.03103
GAE Factor	0.9529
Learning Rate	0.0007154
Timesteps per Batch	2048
VF Iterations	5